

Paginated, Rate-Limited Batch Email Workflow

Lesson 5 Assessment — Documentation Package

Objective: Build and validate a production-style n8n workflow that retrieves records from a paginated API, aggregates the dataset, processes records in configurable batches, tolerates record-level failures, applies rate limiting, and generates execution metrics.

1. Workflow Architecture Diagram

Manual Trigger → Config → Init State → Prepare Page Request → Fetch Page → Accumulate & Check → More Pages? → Combine All Pages → Split In Batches → Send Email (Simulated) → Check Success → Rate Limit Pause → Summary

The workflow is organized into reusable configuration, paginated retrieval, dataset aggregation, controlled batch processing, failure classification, rate limiting, and execution-summary stages.

2. Pagination Design

The workflow retrieves users from the JSONPlaceholder API using page-based pagination. The configured page size is 10. Page retrieval uses the `_page` and `_limit` parameters. The More Pages? conditional controls whether another page should be requested or whether pagination should stop and the accumulated records should move to the aggregation stage.

Configured `pageSize`: 10

Pagination stop behavior: Continue while records are returned; stop when the dataset is exhausted.

3. Batch Processing Design

Records from the retrieved pages are combined into one processing dataset. Split In Batches then processes records in controlled groups. The reusable batch size is 5, allowing the workflow to process records in groups of five and handle a smaller final batch when the total record count is not evenly divisible.

Configured `batchSize`: 5

4. Simulated Email Processing & Error Handling

Each record is processed through a simulated email HTTP request using record-level information such as name and email. Continue On Fail behavior allows an individual processing error to be preserved and classified without terminating the complete workflow. Check Success classifies outcomes as successful or failed for reporting.

Execution evidence: The tested run reported 10 total records, with 8 successful outcomes and 2 failed outcomes. This produces a consistent total of 10 and demonstrates record-level failure tolerance.

5. Rate Limiting Strategy

The Rate Limit Pause Wait stage pauses between batch cycles. The pause duration is controlled through the reusable `waitSeconds` configuration rather than a hard-coded value, making the workflow easier to adjust.

Configured `waitSeconds`: 2 seconds

6. Execution Summary Design

The Summary stage reports operational metrics for the completed run. Required metrics are `totalRecords`, `totalSucceeded`, and `totalFailed`. The validated execution produced:

Metric	Validated Result
<code>totalRecords</code>	10

Metric	Validated Result
totalSucceeded	8
totalFailed	2

7. Configuration Reference

Setting	Value / Design
pageSize	10
batchSize	5
waitSeconds	2
API	JSONPlaceholder users endpoint
Pagination parameters	_page and _limit
Batch processor	Split In Batches
Failure tolerance	Continue On Fail
Classification	Check Success
Rate limiting	Wait-based pause
Summary metrics	totalRecords, totalSucceeded, totalFailed

8. End-to-End Test Plan

Test Area	Expected Evidence
Pagination	Multiple page requests followed by correct stop condition.
Aggregation	All retrieved records available as one processing dataset.
Batching	Records processed in groups of 5, with smaller final batch when applicable.
Successful email	Successful record classified as Sent / successful.
Failed email	Individual failure classified without terminating the workflow.
Rate limiting	2-second pause between batch cycles.
Summary	$\text{totalRecords} = \text{totalSucceeded} + \text{totalFailed}$.
End-to-end execution	Complete workflow finishes successfully with summary output.

9. Workflow Governance & Documentation Standards

Use descriptive node names that communicate purpose, such as Config, Fetch Page, More Pages?, Combine All Pages, Split In Batches, Send Email (Simulated), Check Success, Rate Limit Pause, and Summary. Sticky-note annotations should explain pagination, batch control, failure tolerance, rate limiting, and summary behavior.

10. Repository & Submission Package

Public GitHub repository: Include the exported n8n workflow JSON, documentation, test results, execution screenshots, and setup instructions.

Loom/YouTube: Demonstrate pagination execution, page-loop behavior, dataset aggregation, batch processing, rate limiting, failure handling, and final summary generation.

Documentation: This package covers architecture, pagination, batch processing, error handling, rate limiting, and execution summary design.

11. Setup Instructions

1. Import the exported workflow JSON into n8n.
2. Confirm internet access to the JSONPlaceholder API.
3. Verify `pageSize=10`, `batchSize=5`, and `waitSeconds=2` in Config.
4. Run the workflow with Manual Trigger.
5. Inspect pagination, aggregation, batching, failure classification, rate limiting, and Summary.
6. Export the latest workflow JSON after the final validated change.
7. Add the JSON, screenshots, documentation, and setup instructions to the public GitHub repository.

12. LMS Description — Suggested Comments

Challenges faced: Building and validating the pagination loop, controlled batch processing, and record-level failure handling required end-to-end execution testing.

Resolution: The workflow was generated and tested in n8n, with a validated execution producing 10 records, 8 successful outcomes, and 2 failed outcomes while completing the workflow.

Assumptions: JSONPlaceholder is used as the paginated source API and the email stage is simulated through HTTP processing as specified by the assessment.

Known limitations: The email processing is simulated rather than connected to a production email provider.

Additional improvements: Configuration values are reusable, rate limiting is controlled through waitSeconds, and summary metrics provide operational visibility.

Prepared for Lesson 5 — Paginated, Rate-Limited Batch Email Workflow Assessment